Running Jobs

Important

In the summer/fall of 2025, Delta is transitioning from a RedHat 8 based OS stack to a RedHat 9 based OS stack. During the transition, some nodes will have the "old" RH8 stack, while some nodes will have the new RH9 stack.

There are two different versions of the documentation while this is going on. **You are currently looking at pages for the new default RH9 stack.** For documentation on the old RH8 stack, go to the Red Hat 8 page.

To see what's new in Red Hat 9 and what you have to change to transition, please see the Early User Info page.

Accessing the Compute Nodes

Delta implements the Slurm batch environment to manage access to the compute nodes. Use the Slurm commands to run batch jobs or for interactive access (an "interactive job") to compute nodes. See the Slurm quick start guide for an introduction to Slurm. There are multiple ways to access compute nodes on Delta.

- Batch scripts (`sbatch`) or Interactive (`srun` , `salloc`), which is right for me?
  - sbatch: Use batch scripts for jobs that are debugged, ready to run, and don't require interaction. Sample Slurm batch job scripts are provided in the Sample Scripts section. For mixed resource heterogeneous jobs see the Slurm job support documentation. Slurm also supports job arrays for easy management of a set of similar jobs, see the Slurm job array documentation for more information.
  - srun: `srun` will run a single command through Slurm on a compute node. `srun` blocks, it will wait until Slurm has scheduled compute resources, and when it returns, the job is complete. `srun` can be used to launch a shell to get interactive access to compute node(s), this is an "interactive job". The one thing you can't do in an interactive job created by `srun` is to run srun commands; if you want to do that, use `salloc` .

    - salloc: Also interactive, use `salloc` when you want to reserve compute resources for a period of time and interact with them using multiple commands. Each command you type after your `salloc` session begins will run on the login node if it is just a normal command, or on your reserved compute resources if prefixed with `srun`. Type `exit` when finished with a `salloc` allocation if you want to end it before the time expires.
- Open OnDemand provides compute node access via JupyterLab, VSCode Code Server, and the noVNC Desktop virtual desktop.
- Direct `ssh` access to a compute node in a **running job** is enabled once the job has **started**. See also, Monitoring a Node During a Job. In the following example, JobID 12345 is running on node cn001

```
$ squeue --job jobid
             JOBID PARTITION     NAME     USER ST       TIME  NODES
NODELIST(REASON)
             12345       cpu     bash   gbauer  R       0:17      1 cn001
```

Then in a terminal session:

```
$ ssh cn001
cn001.delta.internal.ncsa.edu (172.28.22.64)
  OS: RedHat 8.4   HW: HPE   CPU: 128x    RAM: 252 GB
  Site: mgmt  Role: compute
$
```

Partitions (Queues)

Delta Partitions/Queues

Delta Partitions/Queues

Partition/Queue	Node Type	Max Nodes per job	Max Duration	Max Running in Queue/user	Charge Factor
cpu	CPU	ALL	48 hr	TBD	1.0
cpu-interactive	CPU	4	1 hr	TBD	2.0
cpu-preempt	CPU	ALL	48 hr	TBD	0.5

Partition/Queue	Node Type	Max Nodes per job	Max Duration	Max Running in Queue/user	Charge Factor
gpuA100x4* (* this is the default queue, but submit jobs to gpuA100x4)	quad-A100	ALL	48 hr	TBD	1.0
gpuA100x4-interactive	quad-A100	4	1 hr	TBD	2.0
gpuA100x4-preempt	quad-A100	ALL	48 hr	TBD	0.5
gpuA100x8	octa-A100	ALL	48 hr	TBD	1.5
gpuA100x8-interactive	octa-A100	2	1 hr	TBD	3.0
gpuH200x8	octa-H200	1	48 hr	TBD	3.0
gpuH200x8-interactive	octa-H200	1	1 hr	TBD	6.0
gpuA40x4	quad-A40	ALL	48 hr	TBD	0.5
gpuA40x4-interactive	quad-A40	4	1 hr	TBD	1.0
gpuA40x4-preempt	quad-A40	ALL	48 hr	TBD	0.25
gpuMI100x8	octa-MI100	ALL	48 hr	TBD	0.25
gpuMI100x8-interactive	octa-MI100	ALL	1 hr	TBD	0.5

(Note: ALL in the “Max Nodes per job” column means you can assign up to the number of nodes int that partition.)

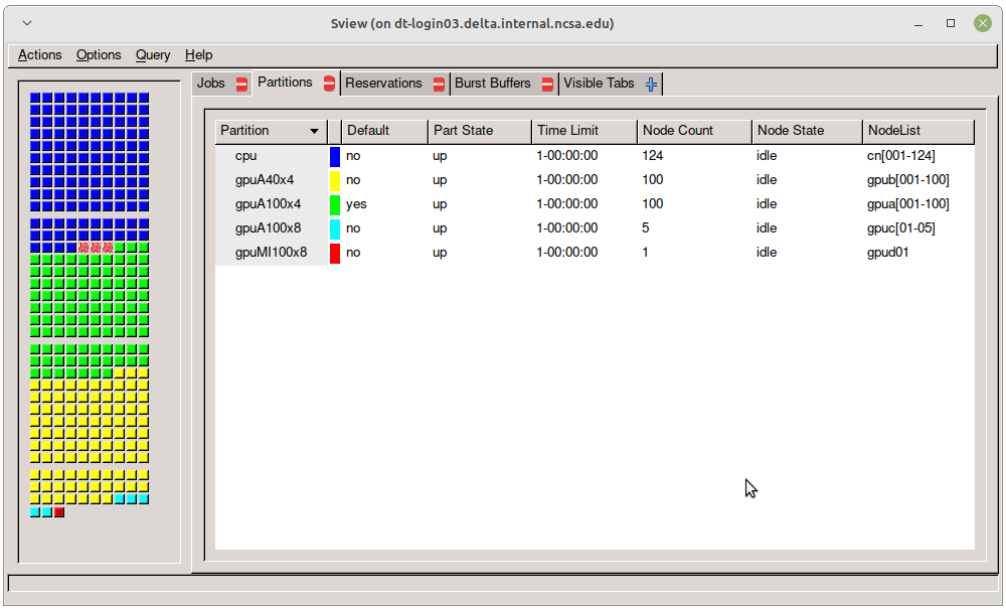
Delta Production Default Partition Values

Delta Default Partition Values

Property	Value
Default Memory per core	1000 MB
Default Wall-clock time	30 minutes

sview

Use `sview` for a GUI of the partitions. See the Slurm - sview documentation for more information.



Job and Node Policies

- The default job requeue or restart policy is set to not allow jobs to be automatically requeued or restarted (as of 12/19/2022). To enable automatic requeue and restart of a job by Slurm, please add the following Slurm directive:

  ```
  --requeue
  ```

  When a job is requeued due to an event like a node failure, the batch script is initiated from its beginning. Job scripts need to be written to handle automatically restarting from checkpoints.

- Node-sharing is the default for jobs. Node-exclusive mode can be obtained by specifying all the consumable resources for that node type or adding the following Slurm options:

  ```
  --exclusive --mem=0
  ```

  GPU NVIDIA MIG (GPU slicing) for the A100 is not supported at this time.

Preemptible Queues

> **Warning**
>
> Preemptible queues are only recommended for jobs that include checkpointing.
>
> If your job code doesn't include checkpointing, then submitting the job to a preempt queue could result in your job being preempted without saved progress/results.

Preemptible queues are available on Delta. See Partitions (Queues) for the partition names, max durations, and charge factors.

On Delta, jobs are allotted a **minimum of 10 minutes** (**PreemptExemptTime**), plus 5 minutes of **GraceTime** if the job has a **SIGTERM** handler.

Slurm Configuration for Preempt Queues

```
# PreemptExemptTime is 10 minutes, so preempt jobs will always get to run at
least 10 minutes
$ scontrol show config | grep PreemptExemptTime
PreemptExemptTime       = 10:00:00

# GraceTime is 5 minutes (300s), a job can potentially run that
# much longer if it handles SIGTERM on its own. SIGKILL arrives at least 5
minutes later.
$ scontrol show partition cpu-preempt | grep -i grace
 DefaultTime=00:30:00 DisableRootJobs=YES ExclusiveUser=NO GraceTime=300
Hidden=NO
```

What Happens When a Job Gets Preempted

1. A preempting job (**job-B**) is allocated resources currently in use by the soon-to-be preempted job (**job-A**)
2. Has **job-A** run for at least 10 minutes (PreemptExemptTime)?
   - If yes, continue to step 3.
   - If no, continue to step 3 after the 10 minutes has elapsed.
3. **job-A** receives **SIGTERM** and **SIGCONT**.
4. 5 minutes later (Delta's **GraceTime** setting on the partition), **job-A** receives another **SIGTERM** and **SIGCONT** plus **SIGKILL** (SIGKILL cannot be handled or caught). SIGKILL is sent after SIGTERM and SIGCONT, but you can't rely on a specific time delay after these signals.

▶ **Preempted Job Example** *(click to expand/collapse)*

Preemption References

There are many online resources to learn more about preemption, checkpointing, signals, and traps; here are a few to get you started.

- Slurm preemption documentation
- PyTorch checkpoint documentation
- TensorFlow checkpoint documentation
- Ubuntu signal man page
- Bash Guide for Beginners - 12.2. Traps
- Python signal documentation

Batch Jobs

Batch jobs are submitted through a *job script* (as in the Sample Scripts) using the `sbatch` command. Job scripts generally start with a series of Slurm *directives* that describe requirements of the job, such as number of nodes and wall time required, to the batch system/scheduler. The rest of the batch script consists of user commands. See Sample Scripts for example batch job scripts.

sbatch

Slurm directives can also be specified as options on the sbatch command line; command line options take precedence over those in the script.

The syntax for sbatch is: `sbatch [list of sbatch options] script_name`. Refer to the sbatch man page for detailed information on the options.

```
$ sbatch tensorflow_cpu.slurm
Submitted batch job 2337924
$ squeue –u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES
NODELIST(REASON)
           2337924 cpu-inter    tfcpu  mylogin  R       0:46      1 cn006
```

Useful Batch Job Environment Variables

Useful Batch Job Environment Variables

Description	Slurm Environment Variable	Detail Description

Description	Slurm Environment Variable	Detail Description
Array JobID	$SLURM_ARRAY_JOB_ID $SLURM_ARRAY_TASK_ID	Each member of a job array is assigned a unique identifier.
Job Submission Directory	$SLURM_SUBMIT_DIR	By default, jobs start in the directory that the job was submitted from. So the `cd $SLURM_SUBMIT_DIR` command is not needed.
JobID	$SLURM_JOB_ID	Job identifier assigned to the job.
Machine(node) list	$SLURM_NODELIST	Variable name that contains the list of nodes assigned to the batch job.

See the sbatch man page for additional environment variables available.

Interactive Jobs

Interactive jobs can be implemented in several ways, depending on what is needed. The following examples start up a bash shell terminal on a CPU or GPU node. (Replace `account_name` with one of your available accounts; these are listed under "Project" when you run the `accounts` command.)

- Single core with 16GB of memory, with one task on a CPU node

```
srun --account=account_name --partition=cpu-interactive \
  --nodes=1 --tasks=1 --tasks-per-node=1 \
  --cpus-per-task=4 --mem=16g \
  --pty bash
```

- Single core with 20GB of memory, with one task on a A40 GPU node

```
srun --account=account_name --partition=gpuA40x4-interactive \
  --nodes=1 --gpus-per-node=1 --tasks=1 \
  --tasks-per-node=16 --cpus-per-task=1 --mem=20g \
  --pty bash
```

Note

For large, compute intensive compilations consider compiling your code on a compute node where you can get more power than on the shared login nodes. You can use srun to start an interactive job which will start a shell on a compute node with access to your files.

srun

The `srun` command initiates an interactive job or process on compute nodes. For example, the following command will run an interactive job in the gpuA100x4 or gpuA40x4 partition with a wall-clock time limit of 30 minutes, using one node and 16 cores per node and 1 GPU. (Replace `account_name` with one of your available accounts; these are listed under "Project" when you run the `accounts` command.)

```
srun -A account_name --time=00:30:00 --nodes=1 --ntasks-per-node=16 \
--partition=gpuA100x4,gpuA40x4 --gpus=1 --mem=16g --pty /bin/bash
```

After entering the command, wait for Slurm to start the job. As with any job, an interactive job is queued until the specified number of nodes is available. Specifying a small number of nodes for smaller amounts of time should shorten the wait time because such jobs will backfill among larger jobs. You will see something like this:

```
$ srun --mem=16g --nodes=1 --ntasks-per-node=1 --cpus-per-task=4 \
--partition=gpuA100x4-interactive,gpuA40x4-interactive --account=account_name
\
--gpus-per-node=1 --time=00:30:00 --x11 --pty /bin/bash
[login_name@gpua022 bin]$  #<-- note the compute node name in the shell prompt
[login_name@gpua022 bin]$ echo $SLURM_JOB_ID
2337913
[login_name@gpua022 ~]$ c/a.out 500
count=500
sum= 0.516221
[login_name@gpua022 ~]$ exit
exit
$
```

When finished, use the `exit` command to end the bash shell on the compute resource and hence the Slurm `srun` job.

salloc

While being interactive like `srun`, `salloc` allocates compute resources for you, while leaving your shell on the login node. Run commands on the login node as usual, use `exit` to end an salloc session early, and use srun with no extra flags to launch processes on the compute resources. (Replace `account_name` with one of your available accounts; these are listed under "Project" when you run the `accounts` command.)

```
$ salloc --mem=16g --nodes=1 --ntasks-per-node=1 --cpus-per-task=2 \
  --partition=gpuA40x4-interactive,gpuA100x4-interactive \
  --account=account_name --time=00:30:00 --gpus-per-node=1
salloc: Pending job allocation 2323230
salloc: job 2323230 queued and waiting for resources
salloc: job 2323230 has been allocated resources
salloc: Granted job allocation 2323230
salloc: Waiting for resource configuration
salloc: Nodes gpub073 are ready for job
$ hostname #<-- on the login node
dt-login03.delta.ncsa.illinois.edu
$ srun bandwidthTest --htod #<-- on the compute resource, honoring your salloc
settings
CUDA Bandwidth Test - Starting...
Running on...

Device 0: NVIDIA A40
Quick Mode

Host to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)        Bandwidth(GB/s)
32000000                     24.5

Result = PASS
$ exit
salloc: Relinquishing job allocation 2323230
```

MPI Interactive Jobs: Use salloc Followed by srun

Interactive jobs are already a child process of `srun`, therefore, one cannot srun (or mpirun) applications from within them. Within standard batch jobs submitted via sbatch, use `srun` to launch MPI codes. For true interactive MPI, use `salloc` in place of `srun` shown above, then "srun my_mpi.exe" after you get a prompt from salloc (`exit` to end the salloc interactive allocation).

▶ **interactive MPI, salloc and srun** *(click to expand/collapse)*

Interactive X11 Support

To run an X11 based application on a compute node in an interactive session, the use of the `--x11` switch with `srun` is needed. For example, to run a single core job that uses 1G of memory with X11 (in this case an xterm) do the following. (Replace `account_name` with one of your available accounts; these are listed under "Project" when you run the `accounts` command.)

```
srun -A account_name  --partition=cpu-interactive \
  --nodes=1 --tasks=1 --tasks-per-node=1 \
  --cpus-per-task=2 --mem=16g \
  --x11  xterm
```

File System Dependency Specification for Jobs

NCSA requests that jobs specify the file system or systems being used to enable response to resource availability issues. All jobs are assumed to depend on the HOME file system. Jobs that do not specify a dependency on WORK (`/projects`) and SCRATCH (`/scratch`) will be assumed to depend only on the HOME (`/u`) file system.

Slurm Feature/Constraint Labels

File System	Feature/Constraint Label	Note
/projects	projects	(same underlying FS as /taiga)
/work/hdd	work	
IME (/ime)	ime	depends on /work/hdd
TAIGA (/taiga)	taiga	(same underlying FS as /projects)

The Slurm constraint specifier and Slurm Feature attribute for jobs are used to add file system dependencies to a job.

Slurm Feature Specification

For already submitted and pending (PD) jobs, please use the Slurm Feature attribute as follows:

```
$ scontrol update job=JOBID Features="feature1&feature2"
```

For example, to add scratch and ime Features to an already submitted job:

```
$ scontrol update job=713210 Features="scratch&ime"
```

To verify the setting:

```
$ scontrol show job 713210 | grep Feature
   Features=scratch&ime DelayBoot=00:00:00
```

Slurm Constraint Specification

To add Slurm job constraint attributes when submitting a job with sbatch (or with `srun` as a command line argument) use:

```
#SBATCH --constraint="constraint1&constraint2.."
```

For example, to add scratch and ime constraints when submitting a job:

```
#SBATCH --constraint="scratch&ime"
```

To verify the setting:

```
$ scontrol show job 713267 | grep Feature
   Features=scratch&ime DelayBoot=00:00:00
```

Job Management

squeue/scontrol/sinfo

The `squeue`, `scontrol`, and `sinfo` commands display batch job and partition information. The following table has a list of common commands, see the man pages for other available options.

In `squeue` results, if the `NODELIST(REASON)` for a job is `MaxGRESPerAccount`, the user has exceeded the number of cores or GPUs allotted per user or project for a given partition.

Common squeue, scontrol, and sinfo Commands

Slurm Command	Description
`squeue -a`	Lists the status of all jobs on the system.
`squeue -u $USER`	Lists the status of all your jobs in the batch system. Replace `$USER` with your username.
`squeue -j JobID`	Lists nodes allocated to a running job in addition to basic information. Replace `JobID` with the JobID of interest.
`scontrol show job JobID`	Lists detailed information on a particular job. Replace `JobID` with the JobID of interest.
`sinfo -a`	Lists summary information on all the partition.

scancel

The scancel command deletes a queued job or terminates a running job. The example below deletes/terminates the job with the associated JobID.

```
scancel JobID
```

Using Job Dependency to Stagger Job Starts

When submitting multiple jobs, consider using `--dependency` to prevent all of the jobs from starting at the same time. Staggering the job startup resource load prevents system slowdowns. This is especially recommended for Python users

because **multiple jobs that load Python on startup can slow down the system if they are all started at the same time**.

From the `--dependency` man page:

```
-d, --dependency=<dependency_list>

                after:job_id[[+time][:jobid[+time]...]]

After the specified jobs start or are cancelled and 'time' in minutes from job
start or cancellation happens, this job can begin  execution. If  no 'time' is
given then there is no delay after start or cancellation.
```

The following sample script staggers the start of five jobs by 5 minutes each. You can use this script as a template and modify it to the number of jobs you have. The minimum recommended delay time is 3 minutes; 5 minutes is a more conservative choice.

▶ **Sample script that automates the delay dependency** *(click to expand/collapse)*

Monitoring a Node During a Job

You have SSH access to nodes in your running job(s). Some of the basic monitoring tools are demonstrated in the example transcript below. Screen shots are appended so that you can see the output from the tools. Most common Linux utilities are available from the compute nodes (free, strace, ps, and so on).

```
[arnoldg@dt-login03 python]$ squeue -u $USER
             JOBID PARTITION     NAME     USER ST       TIME  NODES
NODELIST(REASON)
           1214412 gpuA40x4- interact  arnoldg  R       8:14      1 gpub045
[arnoldg@dt-login03 python]$ ssh gpub045
gpub045.delta.internal.ncsa.edu (141.142.145.145)
  OS: RedHat 8.4   HW: HPE   CPU: 64x    RAM: 252 GB
Last login: Wed Dec 14 09:45:26 2022 from 141.142.144.42
[arnoldg@gpub045 ~]$ nvidia-smi

[arnoldg@gpub045 ~]$ module load nvtop

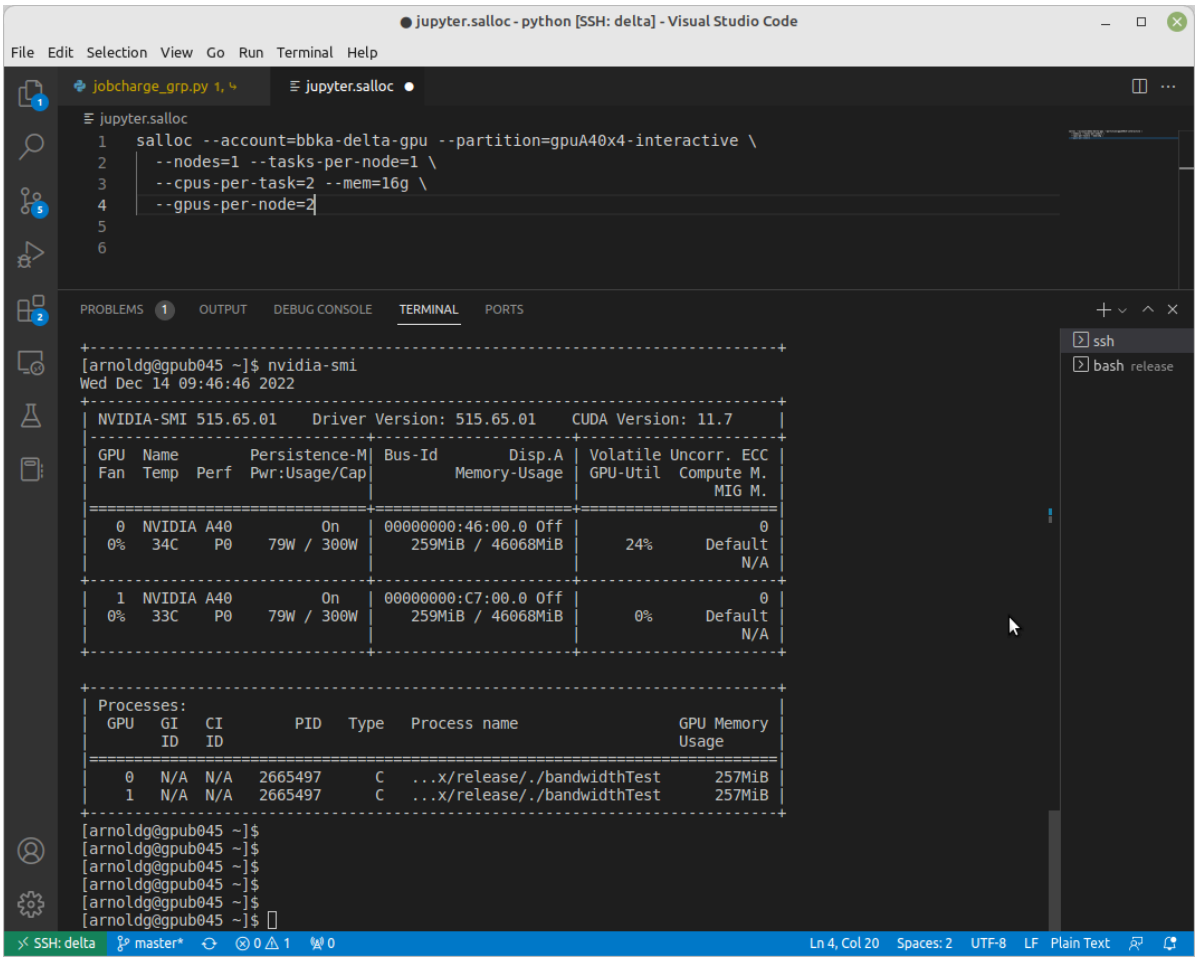
The following have been reloaded with a version change:
1) cuda/11.6.1 => cuda/11.7.0

[arnoldg@gpub045 ~]$ nvtop

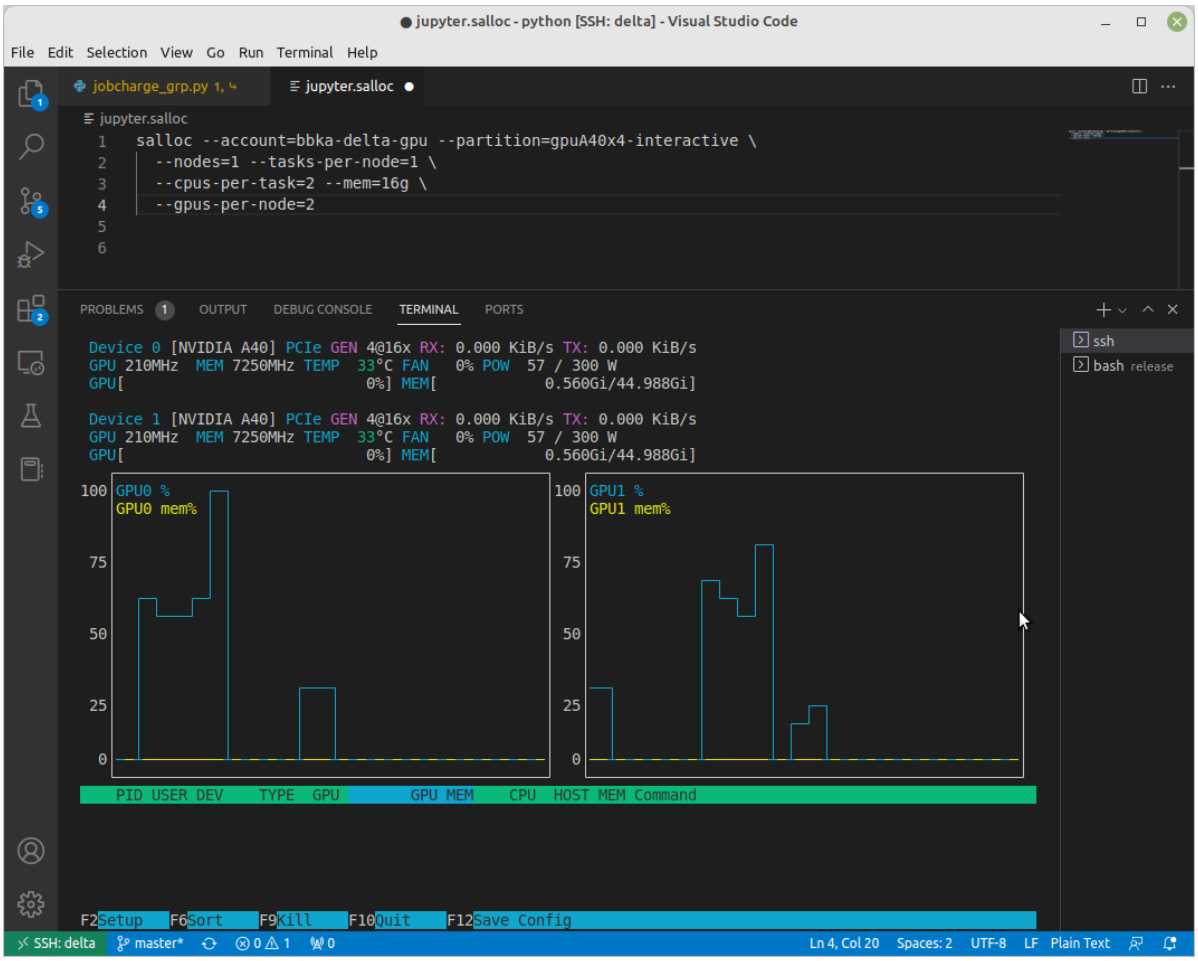
[arnoldg@gpub045 ~]$ module load anaconda3_gpu
[arnoldg@gpub045 ~]$ nvitop

[arnoldg@gpub045 ~]$ top -u $USER
```

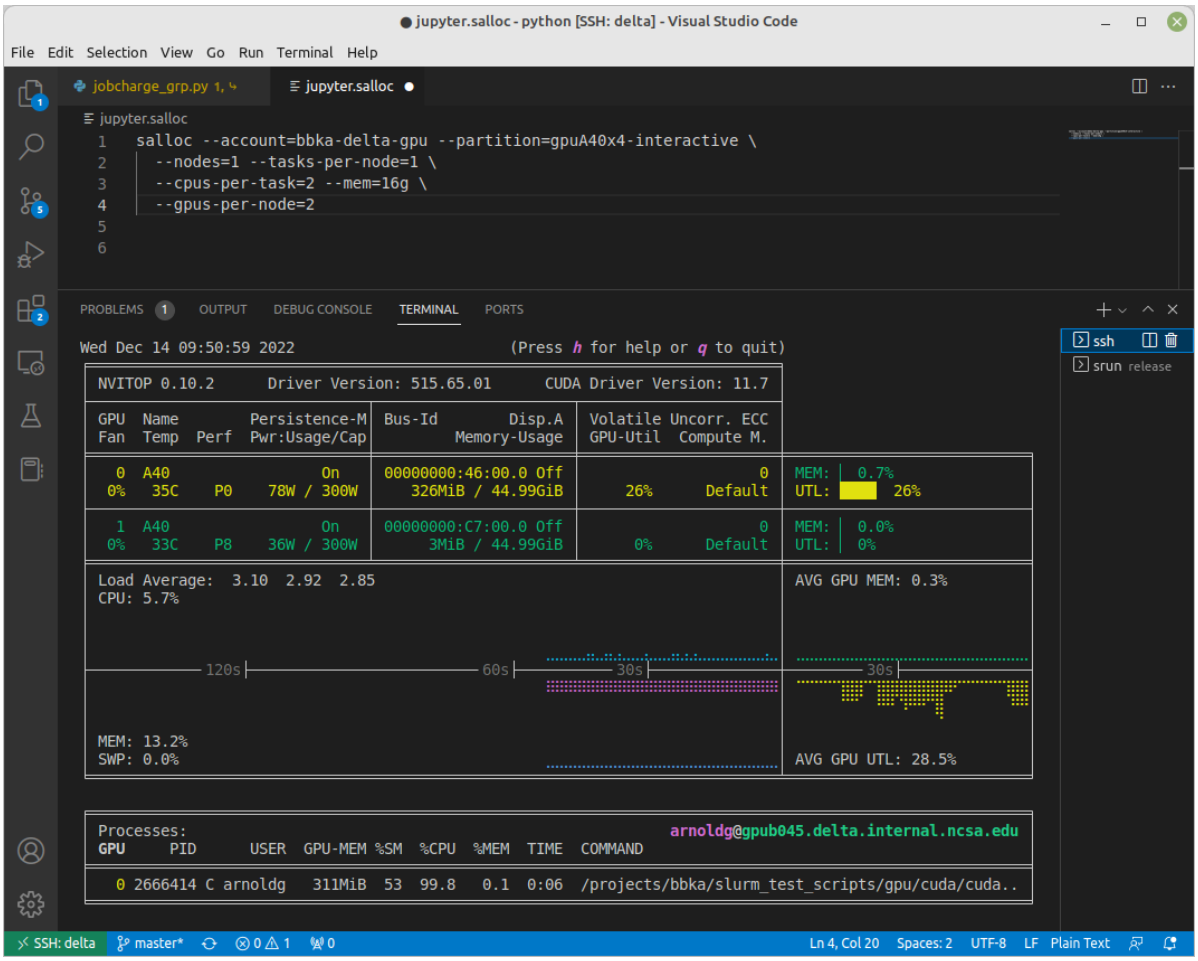
nvidia-smi:



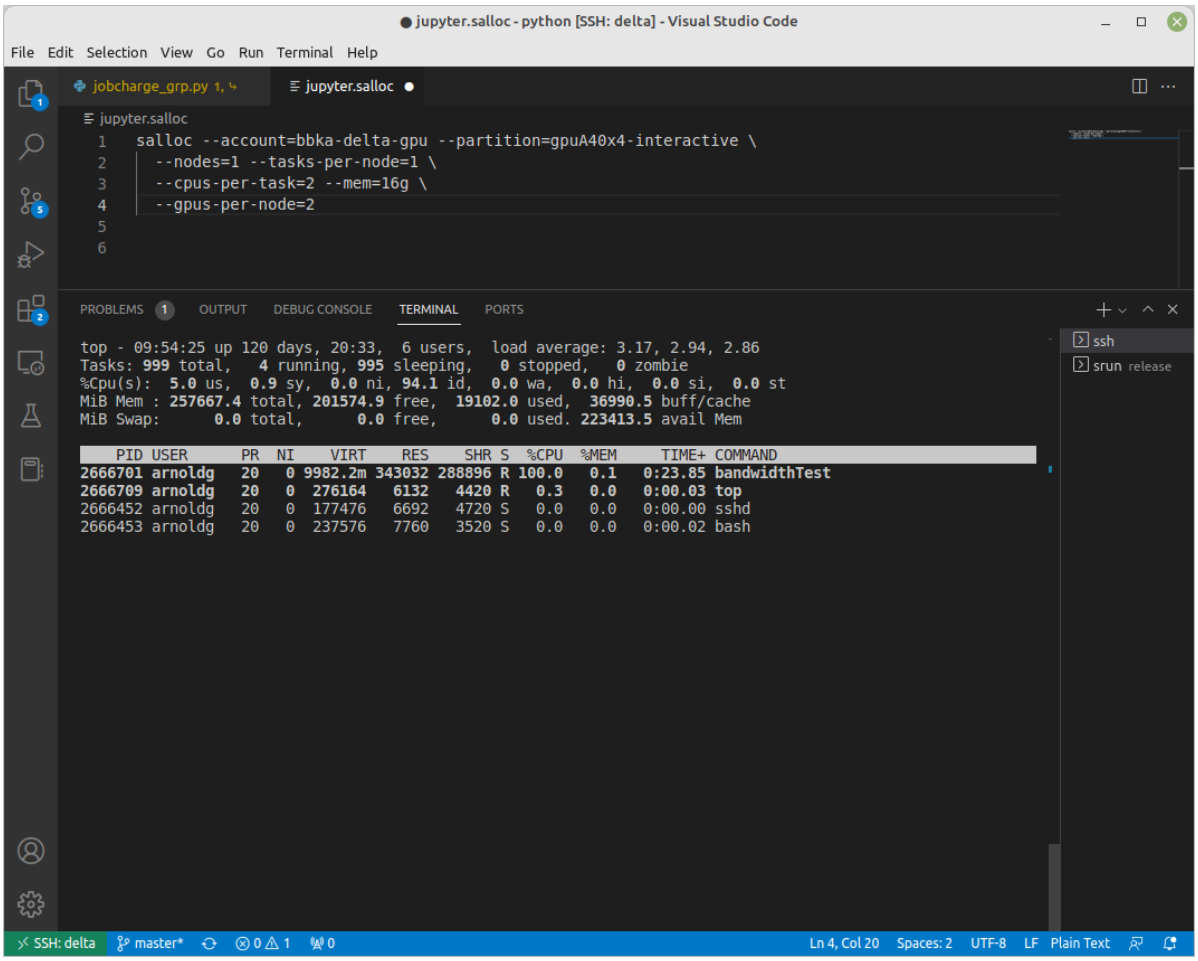
nvtop:


jupyter.salloc - python [SSH: delta] - Visual Studio Code
File Edit Selection View Go Run Terminal Help
jobcharge_grp.py 1,
jupyter.salloc
jupyter.salloc
salloc --account=bbka-delta-gpu --partition=gpuA40x4-interactive \
--nodes=1 --tasks-per-node=1 \
--cpus-per-task=2 --mem=16g \
--gpus-per-node=2
PROBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
ssh
bash release
Device 0 [NVIDIA A40] PCIe GEN 4@16x RX: 0.000 KiB/s TX: 0.000 KiB/s
GPU 210MHz MEM 7250MHz TEMP 33°C FAN 0% POW 57 / 300 W
GPU[0%] MEM[0.560Gi/44.988Gi]
Device 1 [NVIDIA A40] PCIe GEN 4@16x RX: 0.000 KiB/s TX: 0.000 KiB/s
GPU 210MHz MEM 7250MHz TEMP 33°C FAN 0% POW 57 / 300 W
GPU[0%] MEM[0.560Gi/44.988Gi]
100
75
50
25
0
GPU0 %
GPU0 mem%
GPU1 %
GPU1 mem%
PID USER DEV TYPE GPU GPU MEM CPU HOST MEM Command
F2Setup F6Sort F9Kill F10Quit F12Save Config
SSH: delta
master*
Ln 4, Col 20
Spaces: 2
UTF-8
LF
Plain Text

nvitop:


jupyter.salloc - python [SSH: delta] - Visual Studio Code
File Edit Selection View Go Run Terminal Help
jobcharge_grp.py 1,
jupyter.salloc
jupyter.salloc
1 salloc --account=bbka-delta-gpu --partition=gpuA40x4-interactive \
2 --nodes=1 --tasks-per-node=1 \
3 --cpus-per-task=2 --mem=16g \
4 --gpus-per-node=2
5
6
PROBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
ssh
srun release
Wed Dec 14 09:50:59 2022
(Press h for help or q to quit)
NVITOP 0.10.2
Driver Version: 515.65.01
CUDA Driver Version: 11.7
GPU Name Persistence-M Bus-Id Disp.A Volatile Uncorr. ECC
Fan Temp Perf Pwr:Usage/Cap Memory-Usage GPU-Util Compute M.
0 A40 On 00000000:46:00.0 Off 0
0% 35C P0 78W / 300W 326MiB / 44.99GiB 26% Default
MEM: 0.7%
UTL: 26%
1 A40 On 00000000:C7:00.0 Off 0
0% 33C P8 36W / 300W 3MiB / 44.99GiB 0% Default
MEM: 0.0%
UTL: 0%
Load Average: 3.10 2.92 2.85
CPU: 5.7%
120s
60s
30s
30s
MEM: 13.2%
SWP: 0.0%
AVG GPU MEM: 0.3%
AVG GPU UTL: 28.5%
Processes:
arnoldg@gpub045.delta.internal.ncsa.edu
GPU PID USER GPU-MEM %SM %CPU %MEM TIME COMMAND
0 2666414 C arnoldg 311MiB 53 99.8 0.1 0:06 /projects/bbka/slurm_test_scripts/gpu/cuda/cuda..
SSH: delta
master*
Ln 4, Col 20
Spaces: 2
UTF-8
LF
Plain Text

top -u $USER:



Sample Scripts

Serial Jobs on CPU Nodes

▼ **serial example script** *(click to expand/collapse)*

```
$ cat job.slurm
#!/bin/bash
#SBATCH --mem=16g
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=4    # <- match to OMP_NUM_THREADS
#SBATCH --partition=cpu      # <- or one of: gpuA100x4 gpuA40x4 gpuA100x8
gpuMI100x8
#SBATCH --account=account_name    # <- match to a "Project" returned by the
"accounts" command
#SBATCH --job-name=myjobtest
#SBATCH --time=00:10:00      # hh:mm:ss for the job
#SBATCH --constraint="scratch"
#SBATCH -e slurm-%j.err
#SBATCH -o slurm-%j.out
### GPU options ###
##SBATCH --gpus-per-node=2
##SBATCH --gpu-bind=none     # <- or closest
##SBATCH --mail-user=you@yourinstitution.edu
##SBATCH --mail-type="BEGIN,END" See sbatch or srun man pages for more email
options


module reset # drop modules and explicitly load the ones needed
             # (good job metadata and reproducibility)
             # $WORK and $SCRATCH are now set
module load python  # ... or any appropriate modules
module list  # job documentation and metadata
echo "job is starting on `hostname`"
srun python3 myprog.py
```

MPI on CPU Nodes

▶ **mpi example script** *(click to expand/collapse)*

OpenMP on CPU Nodes

▶ **openmp example script** *(click to expand/collapse)*

Hybrid (MPI + OpenMP or MPI+X) on CPU Nodes

▶ **mpi+x example script** *(click to expand/collapse)*

4 GPUs Together on a Compute Node

▶ **4 gpus example script** *(click to expand/collapse)*

1 GPU on a Compute Node

▶ **1 gpu example script** *(click to expand/collapse)*

PyTorch Multi-Node

▶ **pytorch multi-node example script** *(click to expand/collapse)*

Parametric / Array / HTC Jobs

- Not yet implemented.